Stage 1: Input - Malware Samples

Table of Contents

Overview

Stage 1 is the first stage of the malware analysis pipeline. Its primary responsibility is to prepare malware samples for further analysis by extracting essential metadata and ensuring the integrity of the input files. This stage sets the foundation for subsequent stages by validating files, extracting basic features, and saving metadata for future reference.

Key Features

1. Metadata Extraction

- Extracts basic information about each malware sample, including:
  - **File Path**: The location of the sample in the file system.
  - **Name**: The name of the file.
  - **Size**: The size of the file in bytes.
  - **File Type**: The MIME type of the file (e.g., `application/x-executable`).
  - **Encoding**: The encoding format of the file (e.g., `binary`, `utf-8`).
  - **Hashes**: MD5, SHA1, and SHA256 hashes for file integrity and validation.
  - **Timestamp**: The last modification time of the file.

2. Batch and Single File Processing

- **Batch Mode**: Processes multiple samples located in a directory, iterating through each file and extracting metadata.
- **Single File Mode**: Processes a single sample when specified.

3. Hash Tree Integration

- Integrates the `HashTree` structure to store and manage file hashes across pipeline stages.
- Ensures that hashes are calculated accurately and stored for future validation.

4. File Type and Encoding Analysis

- Uses the `magic` module to determine the MIME type and encoding of each file.
- Flags files with unknown or unsupported formats.

5. Metadata Storage

- Saves extracted metadata to a JSON file for future use.
- Ensures that metadata is stored in a structured format for easy integration with subsequent pipeline stages.

6. Error Handling

Stage 1 incorporates robust error handling mechanisms to ensure smooth execution and provide meaningful feedback in case of issues. The following types of errors are handled gracefully:

Missing Files or Directories

- Alerts the user if the specified input file or directory does not exist.
- Example:

```
if not Path(self.input_file).exists():
    self.console.print("[bold red]Error:[/bold red] Directory not
found.")
    return False
```

File Type or Encoding Analysis Failures

- Uses the `magic` module to analyze file type and encoding.
- If the analysis fails, the program flags the issue and provides detailed error feedback.
- Example:

```
try:
    file_type = mime_detector.from_file(input_file)
```

```
    encoding = encoding_detector.from_file(input_file)
except Exception as e:
    self.console.print(f"[bold red]Error:[/bold red] Failed to analyze
file '{input_file}': {str(e)}")
    return False
```

Metadata Storage Issues

- Ensures that metadata is saved correctly to the specified output directory.
- Handles cases where:
  - The output directory does not exist.
  - The program lacks permissions to create or write files.
- Example:

```
try:
    if not Path(self.output_dir).exists():
        Path(self.output_dir).mkdir(parents=True, exist_ok=True)
except Exception as e:
    self.console.print(f"[bold red]Error:[/bold red] Failed to create
output directory '{self.output_dir}': {str(e)}")
    return False
```

No Samples to Process

- Alerts the user if no valid samples are found for processing.
- Example:

```
if not self.samples:
    self.console.print("[bold red]Error:[/bold red] No samples to save.
Metadata file will not be created.")
    return False
```

Unknown or Unexpected Errors

- Catches and logs any unexpected errors during execution, providing a fallback mechanism.
- Example:

```
try:
    # Main processing logic
except Exception as e:
```

```
        self.console.print(f"[bold red]Critical Error:[/bold red] {str(e)}")
        return False
```

Error Logging

To improve debugging and transparency, consider implementing an error logging mechanism:

- Write errors to a log file for later analysis.
- Example:

```
import logging

logging.basicConfig(filename="pipeline_errors.log", level=logging.ERROR)
logging.error(f"Error occurred: {str(e)}")
```

Best Practices for Error Handling

- User-Friendly Feedback:
  - Use clear and concise error messages that help users understand what went wrong and how to fix it.
- Fallback Mechanisms:
  - Ensure that the pipeline can recover gracefully from minor issues (e.g., skipping invalid files).
- Documentation:
  - Provide detailed documentation for common errors and troubleshooting steps.

Future Enhancements

- Dynamic Retry Mechanism:
  - Automatically retry failed operations (e.g., re-analyzing a file) a configurable number of times.
- Advanced Logging:
  - Include timestamps, pipeline stage information, and error severity levels in log files.
- Error Visualization:
  - Generate reports or dashboards summarizing errors for easier debugging.

Example Error Flow

1. A user specifies a directory for batch processing.
2. The program checks if the directory exists:
   1. If it doesn't, an error is displayed: [bold red]Error:[/bold red] Directory not found.
   2. If it does, the pipeline proceeds to analyze the files.
3. During file analysis, the magic module encounters an unsupported file type:
   - The error is flagged: [bold red]Error:[/bold red] Failed to analyze file 'sample.txt': Unsupported file type.
   - Supported file types are listed in the documentation for reference.

4. The pipeline attempts to save metadata but lacks permissions:
    - The error is logged and displayed: [bold red]Error:[/bold red] Failed to write metadata to file: Permission denied.
    - The user is advised to check file permissions and retry the operation.

Workflow

Step 1: Initialize Stage

- The `Stage1` class is instantiated with the following parameters:
    - `input_file`: Path to the input file or directory containing malware samples.
    - `output_dir`: Directory where the processed samples will be saved.
    - `batch_mode`: Flag to enable batch processing of multiple samples.
    - `save_metadata`: Flag to enable saving metadata to a JSON file.

Step 2: Process Samples

- **Batch Mode**:
    - Iterates through all files in the specified directory.
    - Extracts metadata for each file and stores it in the `samples` list.
- **Single File Mode**:
    - Processes a single file by extracting metadata and storing it in the `samples` list.

Step 3: Analyze File

- Extracts metadata for each file using the `magic` module:
    - **File Type**: MIME type of the file.
    - **Encoding**: Encoding format of the file.

Step 4: Calculate File Hashes

- Generates MD5, SHA1, and SHA256 hashes for each file using the `calculate_file_hashes` function.
- Stores hashes in the `HashTree` structure for validation and tracking.

Step 5: Save Metadata

- Saves metadata for all processed samples to a JSON file in the specified output directory.
- Ensures that the metadata is structured and includes all extracted features.

Error Handling

Stage 1 includes robust error handling mechanisms to ensure smooth execution:

- **File Not Found**: Alerts the user if the input file or directory does not exist.
- **Unsupported File Type**: Flags files with unknown MIME types or encodings.
- **Metadata Save Failure**: Notifies the user if metadata cannot be saved due to directory or file issues.

Classes and Functions

Stage1 Class

- Responsible for orchestrating the processing of malware samples.
- Key methods:
  - `process_samples`: Main method for processing samples in batch or single mode.
  - `process_file`: Extracts metadata for a single file.
  - `process_batch`: Processes multiple samples in a directory.
  - `analyze_file`: Uses the `magic` module to analyze file type and encoding.
  - `save_metadata_to_file`: Saves metadata to a JSON file.

Sample Class

- A dataclass representing a malware sample with detailed metadata.
- Attributes include:
  - `file_path`: Path to the sample file.
  - `name`: Name of the file.
  - `size`: File size in bytes.
  - `file_type`: MIME type of the file.
  - `encoding`: Encoding format of the file.
  - `hashes`: A dictionary containing hash trees for each stage.
  - `timestamp`: Last modification time of the file.
  - `data`: Nested dictionary storing extracted features (e.g., strings, headers, entropy).

HashTree Class

- A dataclass designed to store hash information for each pipeline stage.
- Attributes include:
  - `stage_id`: The stage number (e.g., Stage 1, Stage 2).
  - `MD5`: The MD5 hash of the file.
  - `SHA1`: The SHA1 hash of the file.
  - `SHA256`: The SHA256 hash of the file.

The `HashTree` class is used to store hash information for each file at each stage of the pipeline, ensuring that the integrity of the data is maintained
througout the entire analysis process and a backtrace of the hashes can be performed if needed or to compare the results of different stages to results of
a manual analysis for example.

It is also used to store the hashes of the files in a structured way, so that they can be easily accessed and compared with the hashes of the same file in
other stages of the pipeline. If the hashes of a file in a later stage do not match the hashes of the same file in an earlier stage, it is an indication that
the file has been tampered with or that the analysis results are not reliable and therefore maybe should be discarded or get a different treatment.

Helper Functions

1. `init_sample_tree()`:
    - Initializes a dictionary to store empty hash trees for each stage.
    - Example structure:

```
{
    "stage1": HashTree(),
    "stage2": HashTree(),
    "stage3": HashTree()
}
```

2. `calculate_file_hashes(stage_nr, file_path)`:
    - Calculates MD5, SHA1, and SHA256 hashes for the given file.
    - Associates the hashes with -.. .-the specified pipeline stage.
    - Returns a `HashTree` object containing the calculated hashes.

Integration

- Stage 1 uses these imported classes and helper functions to ensure consistency and modularity throughout the pipeline.
- The `HashTree` structure and `Sample` class are passed to subsequent stages, maintaining the integrity of the hashchain and metadata.
- Stage 2 can then use the results of Stage 1 to perform more in-depth analysis on the malware samples.

Output

- **Console Output**: Displays summaries of processed samples, including metadata and hash information.
- **Metadata File**: Saves extracted metadata to a JSON file in the specified output directory.

Example Usage

```
if __name__ == "__main__":
    base_path =
"C:\\Users\\Hendrik.Siemens\\Documents\\SyntaxAnalyst\\pipeline_scripts"
    input_file = os.path.join(base_path, "input")
    output_dir = os.path.join(base_path, "output")
    batch_mode = True
    save_metadata = True

    stage1 = Stage1(input_file=input_file, output_dir=output_dir,
batch_mode=batch_mode, save_metadata=save_metadata)
    stage1.run()
```

TODOs for Stage 1

TODO

Expand on Hash Validation Logic

While you've mentioned hash comparison as part of the pipeline, you could elaborate on how mismatches will be handled.

- For example:
  - Will the pipeline flag mismatched hashes and halt processing?
  - Will there be an option to reprocess files with mismatched hashes?
  - reprocess files with mismatched hashes?

Error Logging Improvements

Stage 1 coveres error handling well, but adding a note about logging errors (e.g., to a file) could make debugging easier. This doesn't need to be implemented right now—just something to keep in mind for later.

Integration with Stage 2

Stage 1 could briefly mention how it`s output (e.g., Sample objects and HashTree) will be used in Stage 2. This helps tie the stages together conceptually.

Visuals

A simple diagram showing the flow of data through Stage 1 (e.g., Input → Metadata Extraction → Hash Calculation → Output) could make the documentation more engaging.

Next Tasks

- [ ] Expand on Hash Validation Logic:
  - [ ] Define Hash Mismatch Handling:
    - [x] `hok: bool = False` Flag for Hash Mismatch Handling between Stages
    - [ ] Implement Hash Comparison Logic:
      - [ ] Compare Hashes between Stages at the End of Each Stage
      - [ ] Flag Mismatched Hashes
      - [ ] Provide Options for Handling Mismatched Hashes:
        - [ ] Add Retry Mechanism for Mismatched Hashes (Optional)
- [ ] Error Handling Improvements:
  - [x] Check if the file is a directory or a file
  - [x] Check if the file is in the allowed file types:
    - [x] Create an empty sample object to keep the pipeline consistent
- [ ] Error Logging Improvements:
  - [ ] Implement Error Logging:
    - [ ] Implement Error Logging
    - [ ] Implement Error Logging

    - [ ] Add Logging to Key Error Points:
        - [ ] Add Logging to Key Error Points
        - [ ] Add Logging to Key Error Points
    - [ ] Document Logging Mechanism:
        - [ ] Document Logging Mechanism
        - [ ] Document Logging Mechanism
    - [ ] Test Error Logging Functionality:
        - [ ] Test Error Logging Functionality
        - [ ] Test Error Logging Functionality
- [ ] Integration with Stage 2
- [ ] Visuals for the Documentation